

Rapport du projet de Programmation Avancée

Adam Ibnouzahir

December 21, 2023

Contents

1	Introduction	1
2	Présentation du jeu	2
2.1	Règles	2
2.2	Organisation	2
3	Présentation du code	2
3.1	Structures	3
3.2	Fonctions	3
3.2.1	Fonctions du Taquin	3
3.2.2	Fonctions du SDL	4
3.2.3	Fonctions complémentaires	5
3.3	Déroulement du programme	5
4	Jeu textuel	5
4.1	Présentation	5
4.2	Déroulement	5
5	Conclusion	5

1 Introduction

Nous avons reproduit le jeu du taquin dans un programme de C et grâce à la bibliothèque SDL. Ce programme nous permet de créer une fenêtre grâce à laquelle on peut jouer au Taquin.

2 Présentation du jeu

2.1 Règles

Les règles du taquin sont les mêmes règles classiques. Il consiste en un plateau classique composé de quatre lignes et quatre colonnes. Ce plateau comporte quinze pavés avec une case libre. Chaque pavé est numéroté avec un nombre unique compris entre 1 et 15.

Les pavés sont disposés sur le plateau d'une manière désordonnée. Le but est de se ramener à la situation où les nombres sont triés de manière croissante. Seuls les pavés adjacents à la case libre peuvent être placés sur cette case.

2.2 Organisation

Il faut d'abord noter que notre programme peut accueillir un argument n qui définira la taille du puzzle initial. Si aucun argument n'est mis, alors n est initialisé à 4.

Pour notre projet, nous avons décidé d'utiliser la librairie SDL pour créer le jeu sur une fenêtre. En lançant le programme, nous avons d'abord un menu textuel qui nous propose de lancer soit un taquin aléatoire de taille n en tapant 1 ou un taquin importé d'un fichier en tapant 2. Il est à noter que le fichier texte doit être de cette forme, comme par exemple pour un taquin de taille 4 :

```
4
X X X X
X X X X
X X X X
X X X X
```

Une fois que l'option a été choisi, une fenêtre s'ouvre et on peut commencer à jouer au taquin. On déplace la case vide grâce aux touches directionnelles. On peut donc déplacer les cases simplement. Lorsque qu'on remet toutes les cases en ordre, une animation de "victoire" s'ensuit où toutes les cases sont affichés en vert une par une.

3 Présentation du code

Nous allons présenter toutes les différentes fonctions du programme, comment elles marchent et nous donnerons le déroulement de notre programme.

3.1 Structures

Présentons les structures du programme :

- `typedef struct Taquin` : Cette structure représente l'intégralité du jeu de taquin.
 - `int ** puzzle` : Tableau 2D représentant le puzzle.
 - `taille` : Dimensions du tableau de taquin.
 - `int vide_y` : Coordonnée y de l'espace vide.
 - `int vide_x` : Coordonnée x de l'espace vide.
- `typedef enum` : Cette liste d'enum représente les directions dans laquelle le joueur peut déplacer l'espace vide. Elle est composée de `HAUT`, `BAS`, `GAUCHE` et `DROITE`.

3.2 Fonctions

3.2.1 Fonctions du Taquin

Ces fonctions sont contenues dans les fichiers `f_taquin.c` et `f_taquin.h`.

- `int ** CreerTab(int n)` : Prend en paramètre un entier `n` et renvoie l'adresse d'un tableau rempli à deux dimensions de taille `n`. Ce tableau alloue des cases dynamiquement. Il contiendra les valeurs du puzzle du taquin. Toutes les cases seront numérotés dans l'ordre à l'exception de la dernière case en bas à droite qui sera la case vide.
- `bool ContientValeur(Taquin * taquin, int valeur)` : Prend en paramètre une adresse de structure Taquin `taquin` et un entier `valeur`. Cette fonction vérifie si le tableau du taquin contient la valeur indiquée. Si c'est le cas, elle renvoie `true`, sinon elle renvoie `false`. Elle est utilisée pour savoir si un taquin importé est valide.
- `Taquin * ImportTaquin(FILE * file)` : Prend en paramètre une adresse de fichier et renvoie une adresse de taquin. La fonction lit le contenu du fichier pour importer le taquin correspondant. Mais si le contenu ne correspond au format demandé ou si le taquin n'est pas vide, la fonction renvoie une adresse vide.
- `Taquin * CreerTaquin(int n)` : Prend en paramètre un entier `n` et renvoie une adresse de taquin. Ce taquin aura un tableau 2D rempli grâce à la fonction `CreerTab` et tous les attributs remplis au préalable.

- `Taquin * CreerTaquinVide(int n)` : Prend en paramètre un entier `n` et renvoie une adresse de taquin. A la différence de la fonction précédente, elle ne fait pas appel à `CreerTab` et a un tableau rempli de zéros. Elle est utilisée pour remplir un taquin importé.
- `void FreeTaquin(Taquin * puzzle)` : Prend en paramètre un entier `n`. Cette fonction permet de vider toute la mémoire allouée par le tableau du taquin et de libérer l'adresse.
- `void BougeTaquin(Taquin * puzzle, TaquinMouvement touche)` : Prend en paramètre une adresse de taquin et un enum `TaquinMouvement`. Cette fonction est importante car elle permettra au joueur de faire bouger le taquin. Par exemple, si on met `HAUT`, la case libre bougera avec la case du haut, si c'est possible, si c'est `GAUCHE`, elle permutera avec la case de gauche.
- `void InitTaquin(Taquin * puzzle, int iter)` : Prend en paramètre une adresse de taquin avec un tableau organisé et un entier `iter`. Cette fonction de mélanger le tableau du taquin en utilisant *iter* fois la fonction `TaquinMouvement`. On utilise cette fonction pour faire en sorte que le puzzle reste soluble.
- `bool VerifTab(Taquin * taquin)` : Prend en paramètre une adresse de taquin. Elle vérifie à chaque mouvement du joueur si le tableau est dans l'ordre. On utilise cette fonction pour vérifier si le joueur a gagné.

3.2.2 Fonctions du SDL

Ces fonctions sont contenues dans les fichiers `f_sdl.c` et `f_sdl.h`.

- `SDL_Window * CreerFenetre(char * title, int width, int height)` : Prend en paramètre une chaîne de caractères `title` et deux entiers `width` et `height`. Cette fonction simplifie la création de fenêtre en créant une fenêtre de dimension `width * height` avec le titre `title`. Renvoie une adresse nulle si elle échoue.
- `SDL_Renderer * CreerRendu(SDL_Window * window)` : Prend en paramètre l'adresse d'une fenêtre. Simplifie la création du rendu sur une fenêtre pour affecter les graphismes de la fenêtre. Renvoie une adresse vide si elle échoue.
- `SDL_Surface * CreerTexte(TTF_Font * font, char * str, SDL_Color textColor)` : Prend en paramètre une chaîne de caractères, une police

TTF et une couleur SDL. Cette fonction permet de créer une surface SDL en écrivant un texte. Elle renvoie la surface. Renvoie une adresse vide si elle échoue.

- `SDL_Texture * CreerTexture(SDL_Renderer * renderer, SDL_Surface * surface)` : Prend en paramètre le rendu de la fenêtre et une surface SDL. La fonction crée une texture SDL pour le rendu pour afficher le texte créé. Renvoie une adresse vide si elle échoue.
- `SDL_InitAll` : Cette fonction permet d'initialiser les bibliothèques SDL `SDL_INIT(SDL_INIT_VIDEO)` et `TTF_Init()`. Quitte le programme si elle échoue.

3.2.3 Fonctions complémentaires

Ces fonctions font appel aux deux autres fichiers.

- `bool AfficherTaquin(Taquin * taquin, SDL_Renderer * renderer, TTF_Font * font)` : Prend en paramètre l'adresse du taquin, le rendu de la fenêtre et la police TTF. Cette fonction très importante fait appel aux autres fonctions pour pouvoir afficher les cases du jeu de taquin et les réactualiser quand le joueur interagit. La case rouge représente l'espace vide du taquin. Quand le joueur remet en ordre le jeu de taquin, la fonction affiche en vert une par une les cases du puzzle pour faire une animation de victoire. Elle renvoie `false` s'il y a un problème.

3.3 Déroulement du programme

4 Jeu textuel

4.1 Présentation

4.2 Déroulement

5 Conclusion